

LATTICE ENCRYPTION

BY SETH JONES

LATTICE BASICS

What is a
lattice?

Connected shape
created by
vectors

Basis Vectors

Two Vectors that
can reach
every point in a
lattice with
coefficients

Motivation

Post-Quantum
digital security

LATTICE DEMONSTRATION

LATTICE ENCRYPTION

Shortest Vector Problem

Given a lattice; find the lattice
point closest to the origin

Closest Vector Problem

Given a random point
Find the closest Lattice point

LATTICE PROBLEM DEMONSTRATION

LEARNING WITH ERRORS (LWE)

Secret Vector - s

public Vector - a

Noise - e

Linear equation of modulo q - $b = \text{dot}(a, s) + e \bmod q$

Multiple samples = (a, b)

QUESTIONS?

PAPERS

A Survey of Polynomial Multiplications for Lattice-Based Cryptosystems

Vincent Hwang  

Lattice-based Cryptography

A survey on the security of the lattice-based
NIST finalists

Koen de Boer¹ and Wessel van Woerden²

ENVIRONMENT CONSTRUCTION

```
import numpy as np

n = 4
q = 97
std_dev = 3

def KeyGen():
    secKey = np.random.randint(-1, 2, n)
    pubVec = np.random.randint(0, q, n)
    noise = np.random.randint(-std_dev, std_dev + 1, n)
    pubKey = (pubVec * secKey + noise) % q
    return (pubVec, pubKey), secKey
```

VERSION 1.0

```
def Encrypt(pubKeyPair, message):
    pubVec, pubKey = pubKeyPair
    randVec = np.random.randint(-1, 2, n)
    noise1 = np.random.randint(-std_dev, std_dev + 1)
    noise2 = np.random.randint(-std_dev, std_dev + 1)
    cipherVec = (pubVec * randVec + noise1) % q
    cipherVal = (np.dot(pubKey, randVec) + noise2 + (q // 2 if message else 0)) % q
    return (cipherVec, cipherVal)
```

VERSION 1.0

```
def Decrypt(secKey, cipherText):  
    cipherVec, cipherVal = cipherText  
    messageEstimate = (cipherVal - np.dot(cipherVec, secKey)) % q  
    return 1 if messageEstimate > q // 4 and messageEstimate < 3 * q // 4 else 0
```

```
for message in [0, 1]:  
    print(f"Message: {message}")  
    pubKeyPair, secKey = KeyGen()  
    pubVec, pubKey = pubKeyPair  
    print(f"Keys:\n    Public Vector: {pubVec}\n    Public Key: {pubKey}\n    Secret Key: {secKey}")  
    cipherText = Encrypt(pubKeyPair, message)  
    cipherVec, cipherVal = cipherText  
    print(f"CipherText:\n    Cipher Vector: {cipherVec}\n    Cipher Value: {cipherVal}")  
    decrypted = Decrypt(secKey, cipherText)  
    print(f"Decrypted Message: {decrypted}")
```


Message: 1

Keys:

Public Vector: [91 24 9 15]

Public Key: [7 2 86 15]

Secret Key: [-1 0 -1 1]

CipherText:

Cipher Vector: [0 73 9 15]

Cipher Value: 50

Decrypted Message: 1

Message: 0

Keys:

Public Vector: [32 19 42 18]

Public Key: [65 77 52 18]

Secret Key: [-1 -1 -1 1]

CipherText:

Cipher Vector: [67 80 44 20]

Cipher Value: 22

Decrypted Message: 0

Message: 1

Keys:

Public Vector: [90 4 32 23]

Public Key: [90 6 64 75]

Secret Key: [1 1 -1 -1]

CipherText:

Cipher Vector: [1 94 66 24]

Cipher Value: 51

Decrypted Message: 1

Message: 0

Keys:

Public Vector: [37 28 83 33]

Public Key: [95 1 14 1]

Secret Key: [0 0 -1 0]

CipherText:

Cipher Vector: [2 2 2 66]

Cipher Value: 1

Decrypted Message: 0

VERSION 1.0 TESTING

```
def TestEncryption(trials=100):  
    for message in [0, 1]:  
        failures = 0  
        for _ in range(trials):  
            pubKeyPair, secKey = KeyGen()  
            cipherText = Encrypt(pubKeyPair, message)  
            recovered = Decrypt(secKey, cipherText)  
            if recovered != message:  
                failures += 1  
        print(f"Message {message}: {failures}/{trials} failed decryptions")
```

```
Message 0: 0/100 failed decryptions  
Message 1: 0/100 failed decryptions
```

```
def EncryptArray(pubKeyPair, messages):  
    return [Encrypt(pubKeyPair, m) for m in messages]  
  
def DecryptArray(secKey, cipherTexts):  
    return [Decrypt(secKey, c) for c in cipherTexts]
```

```
def TestEncryptionArray(trials=100, array_length=5):  
    for _ in range(trials):  
        pubKeyPair, secKey = KeyGen()  
  
        messages = np.random.randint(0, 2, array_length)  
  
        cipherTexts = EncryptArray(pubKeyPair, messages)  
        decrypted = DecryptArray(secKey, cipherTexts)  
  
        failures = sum(m != r for m, r in zip(messages, decrypted))  
  
        print(f"Message: {messages}")  
        print(f"Decrypted: {decrypted}")  
        print(f"Failed decryptions in this trial: {failures}\n")
```

VERSION 1.1 TESTING

```
Message: [1 0 0 1 1]
Decrypted: [1, 0, 0, 1, 1]
Failed decryptions in this trial: 0
```

```
Message: [1 0 1 1 1]
Decrypted: [1, 0, 1, 1, 1]
Failed decryptions in this trial: 0
```

```
Message: [1 0 0 0 1]
Decrypted: [1, 0, 0, 0, 1]
Failed decryptions in this trial: 0
```

```
Message: [1 0 0 1 0]
Decrypted: [1, 0, 0, 1, 0]
Failed decryptions in this trial: 0
```

```
Message: [1 1 1 1 0]
Decrypted: [1, 1, 1, 1, 0]
Failed decryptions in this trial: 0
```

```
Message: [0 0 0 0 1]
Decrypted: [0, 0, 0, 0, 1]
Failed decryptions in this trial: 0
```

```
Message: [0 0 0 0 1]
Decrypted: [0, 0, 0, 0, 1]
Failed decryptions in this trial: 0
```

```
Message: [1 1 1 0 1]
Decrypted: [1, 1, 1, 0, 1]
Failed decryptions in this trial: 0
```


VERSION 1.1 TESTING

$n = 4028$

$q = 2^{**}15$

$\text{std_dev} = 3$

Message: [1 1 1 1]
Decrypted: [1, 1, 1, 1]
Failed decryptions in this trial: 0

Message: [0 1 1 1]
Decrypted: [0, 1, 1, 1]
Failed decryptions in this trial: 0

Message: [1 1 1 0]
Decrypted: [1, 1, 1, 0]
Failed decryptions in this trial: 0

Message: [0 1 0 1]
Decrypted: [0, 1, 0, 1]
Failed decryptions in this trial: 0

Message: [0 0 0 0]
Decrypted: [0, 0, 0, 0]
Failed decryptions in this trial: 0

Message: [1 0 0 1]
Decrypted: [1, 0, 0, 1]
Failed decryptions in this trial: 0

Message: [1 1 0 1]
Decrypted: [1, 1, 0, 1]
Failed decryptions in this trial: 0

Message: [1 1 0 0]
Decrypted: [1, 1, 0, 0]
Failed decryptions in this trial: 0

Message: [1 0 0 0]
Decrypted: [1, 0, 0, 0]
Failed decryptions in this trial: 0

Message: [0 0 1 0]
Decrypted: [0, 0, 1, 0]
Failed decryptions in this trial: 0

VERSION 1.2

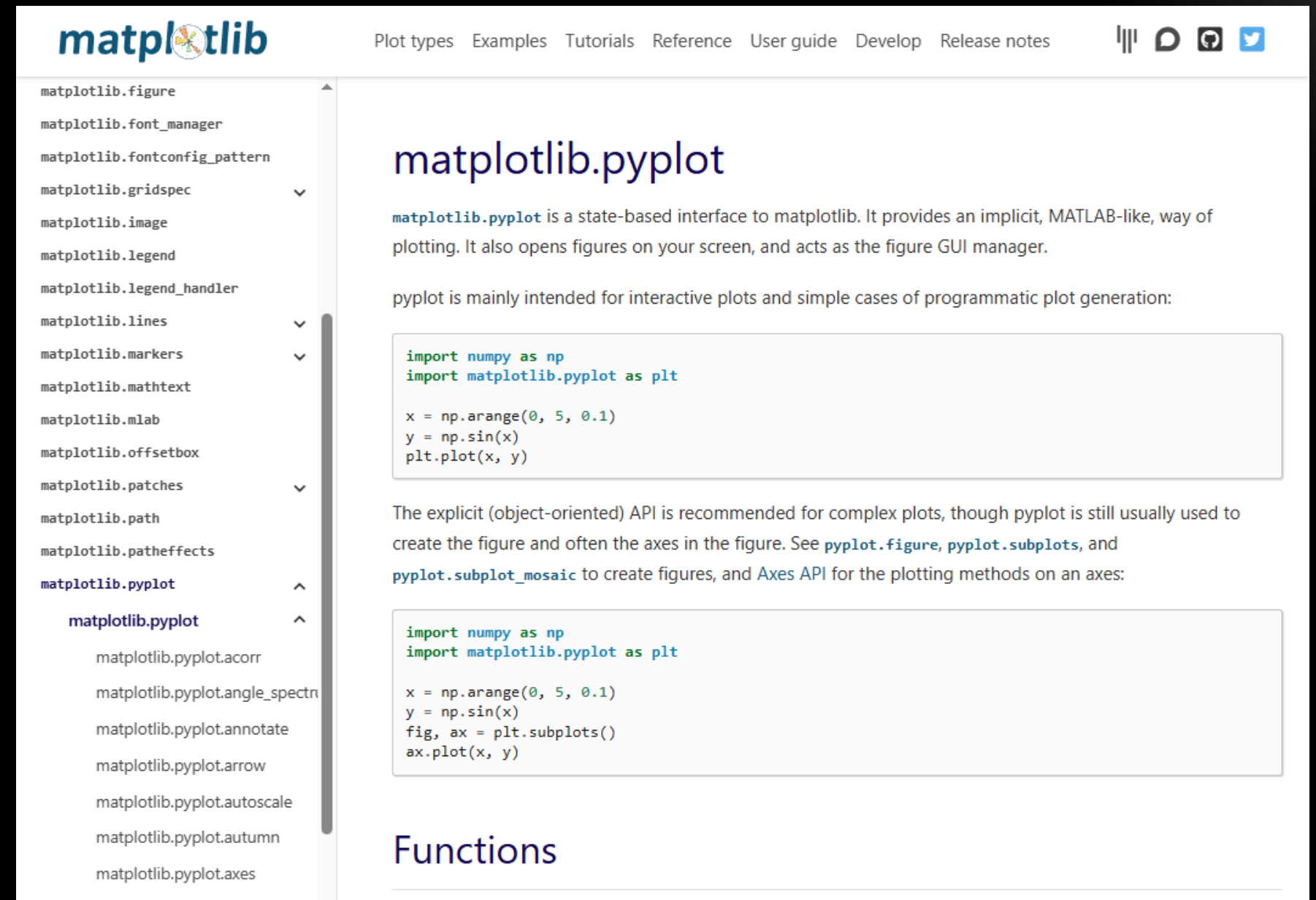
```
noise = np.random.randint(-std_dev, std_dev + 1, n)
```

```
def RandomGaussian(size, stdDev):  
    return np.random.normal(0, stdDev, size=size).round().astype(int)
```

SIMULATING

```
import time
import matplotlib.pyplot as plt
```

MATLAB



The screenshot shows the matplotlib website. The left sidebar contains a list of modules, with `matplotlib.pyplot` selected and expanded. The main content area is titled `matplotlib.pyplot` and contains the following text:

`matplotlib.pyplot` is a state-based interface to matplotlib. It provides an implicit, MATLAB-like, way of plotting. It also opens figures on your screen, and acts as the figure GUI manager.

`pyplot` is mainly intended for interactive plots and simple cases of programmatic plot generation:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
plt.plot(x, y)
```

The explicit (object-oriented) API is recommended for complex plots, though `pyplot` is still usually used to create the figure and often the axes in the figure. See `pyplot.figure`, `pyplot.subplots`, and `pyplot.subplot_mosaic` to create figures, and `Axes API` for the plotting methods on an axes:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
fig, ax = plt.subplots()
ax.plot(x, y)
```

Functions

```
StdDevs = [1, 3, 5, 7, 10]
Ns = [64, 128, 256]
Qs = [2**10, 2**12, 2**15]
MsgLen = 128
```

TESTING 1 ENVIROMENT

```
Results = []

for N in Ns:
    for Q in Qs:
        for StdDev in StdDevs:
            SecretKey, PublicMatrix, PublicVector = KeyGenMatrix(N, Q, StdDev)
            Messages = np.random.randint(0, 2, size=(MsgLen,))

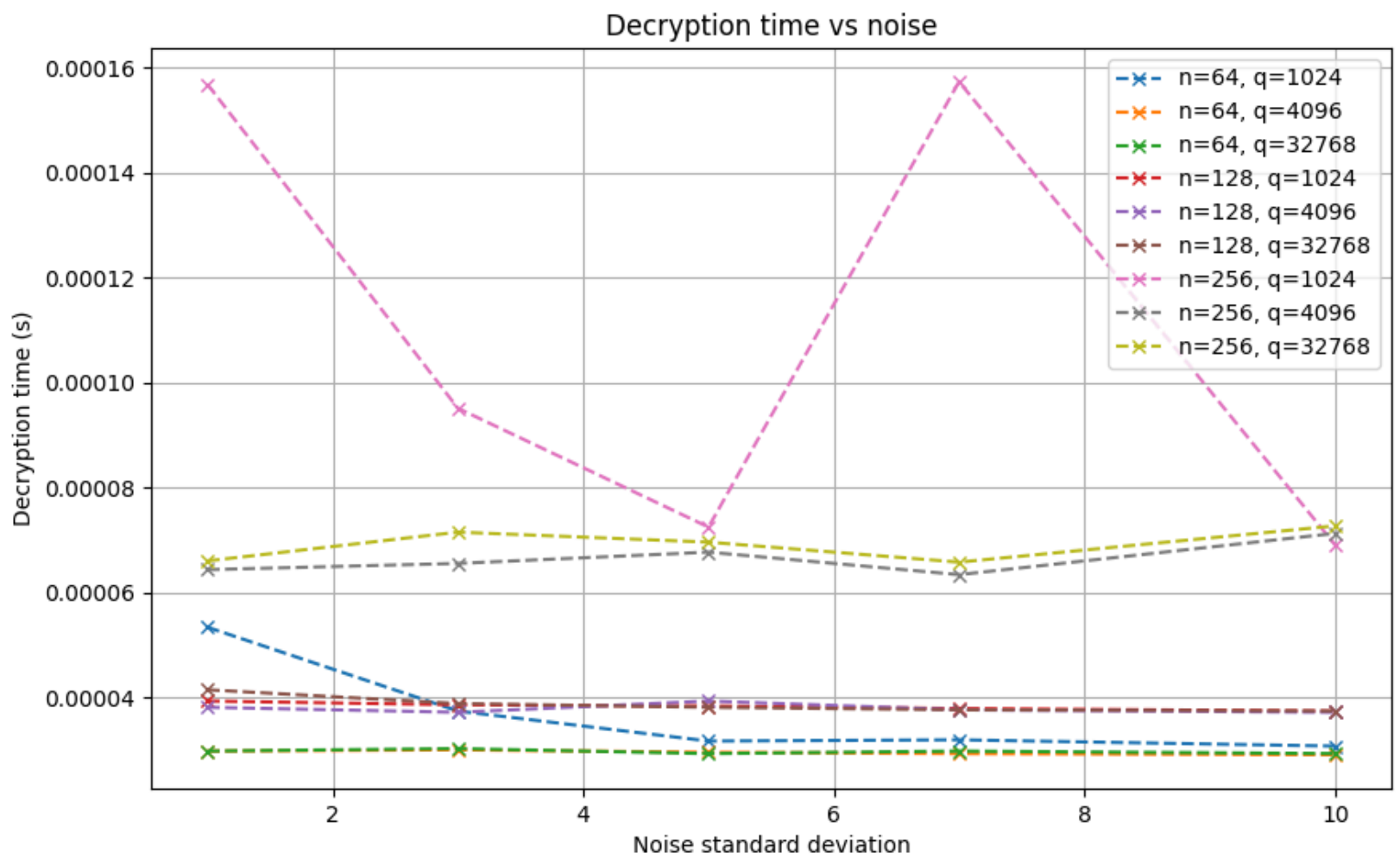
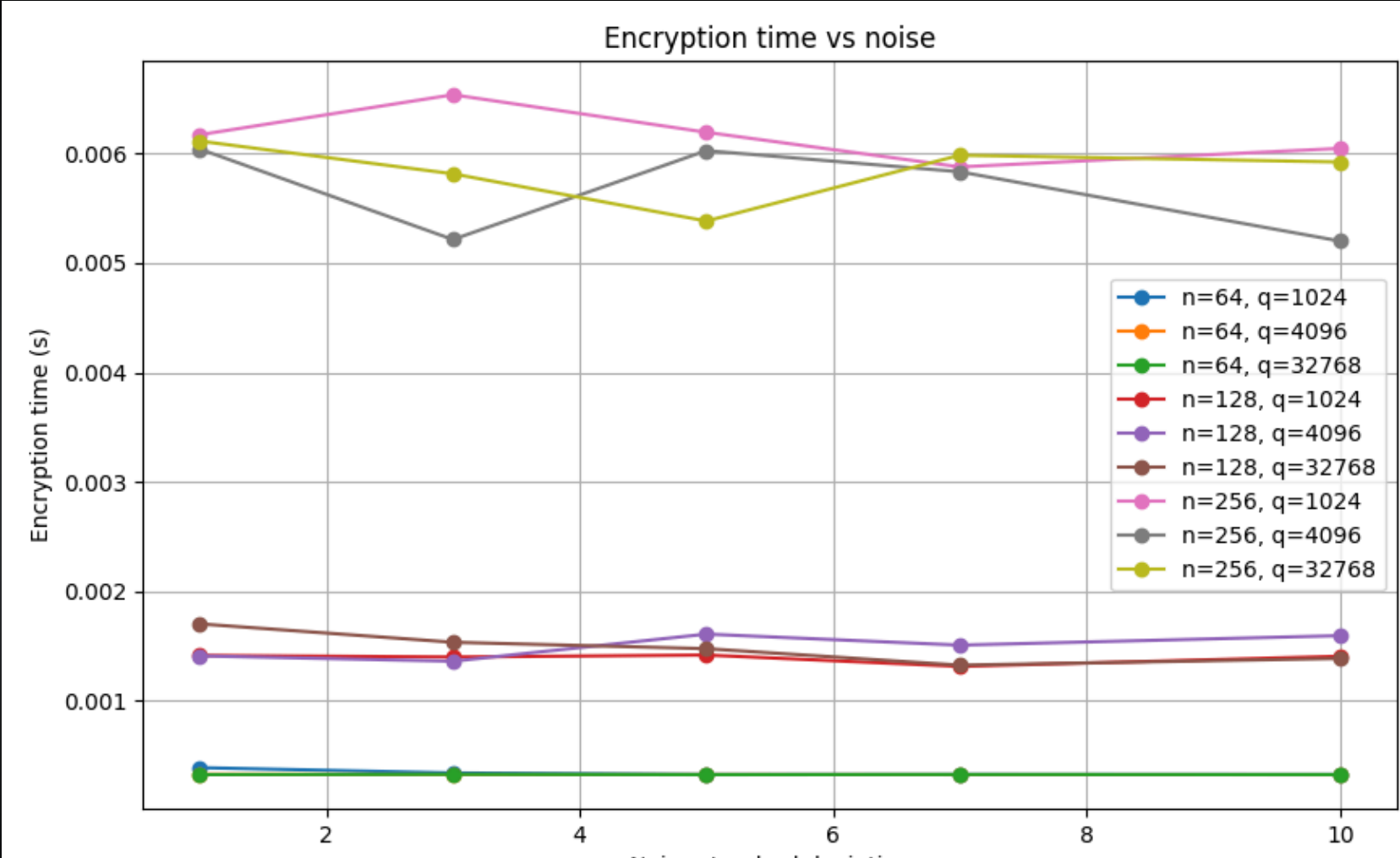
            start_enc = time.time()
            CipherMatrices, CipherValues = EncryptBits(Messages, PublicMatrix, PublicVector, Q)
            EncTime = time.time() - start_enc

            start_dec = time.time()
            DecryptBits(CipherMatrices, CipherValues, SecretKey, Q)
            DecTime = time.time() - start_dec

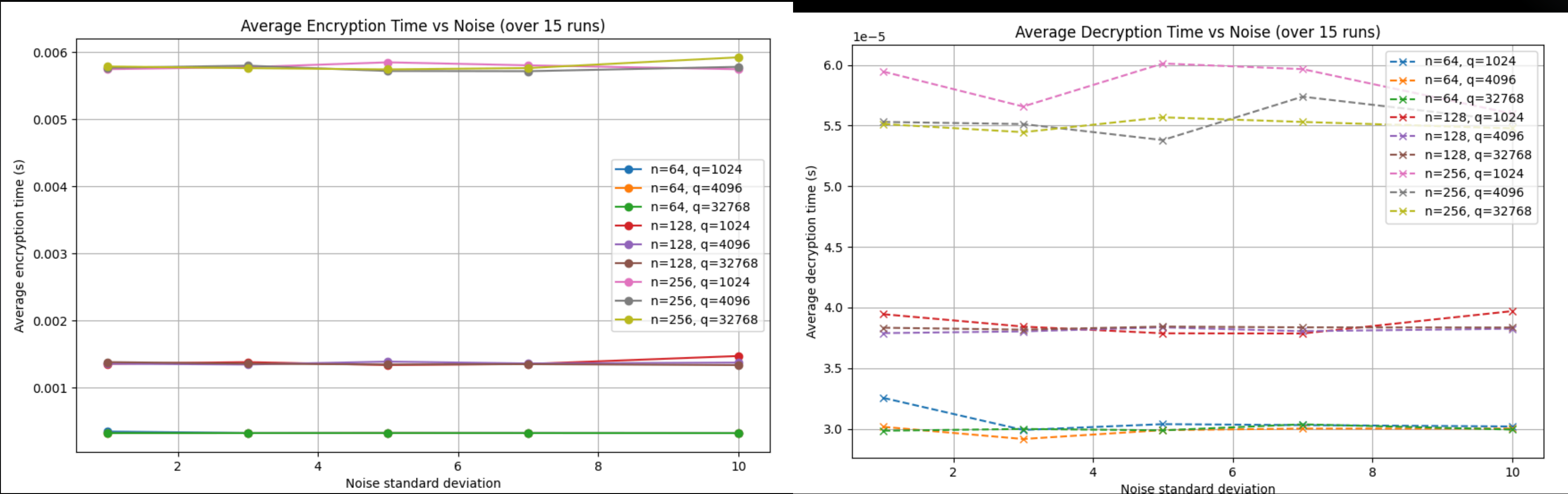
            Results.append({
                'N': N,
                'Q': Q,
                'StdDev': StdDev,
                'EncTime': EncTime,
                'DecTime': DecTime
            })

print(f"N={N}, Q={Q}, Std={StdDev}, Enc={EncTime:.4f}s, Dec={DecTime:.4f}s")
```


TESTING 1



TESTING 1 (REVISED)



```

ns = 256
qs = 3329
msg_len = 128
num_runs = 15

```

```

secKey = np.random.randint(-1, 2, size=(n,))

```

```

def centered_mod(x, q):
    return np.mod(x, q)

```

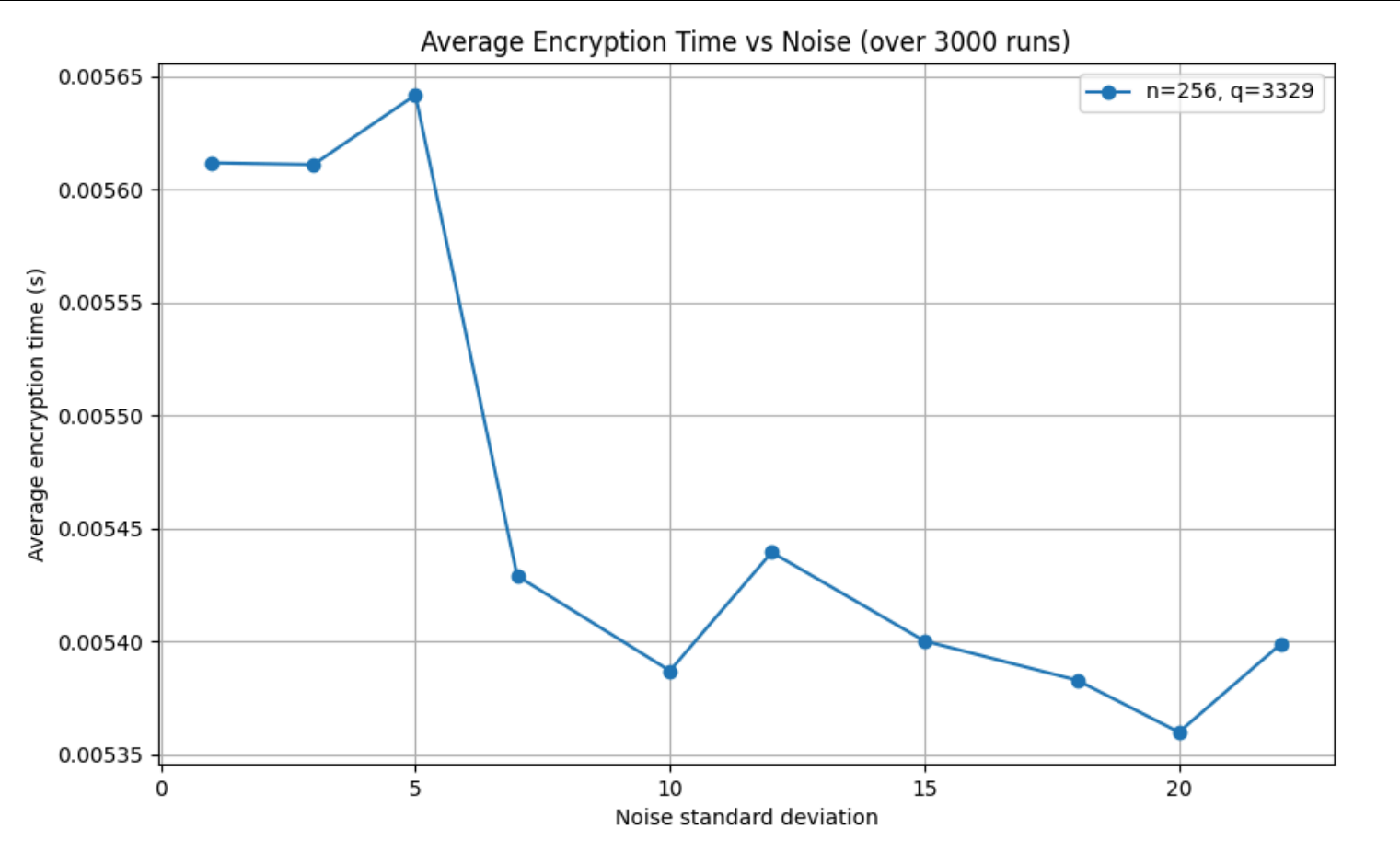
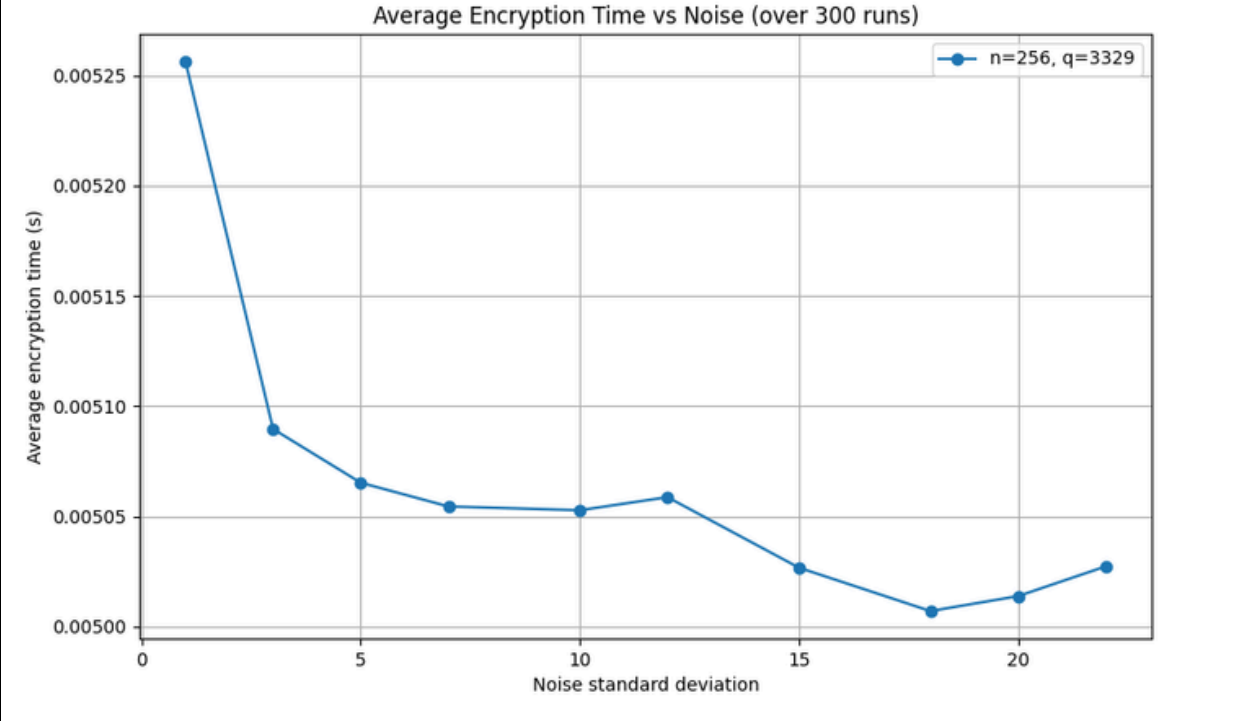
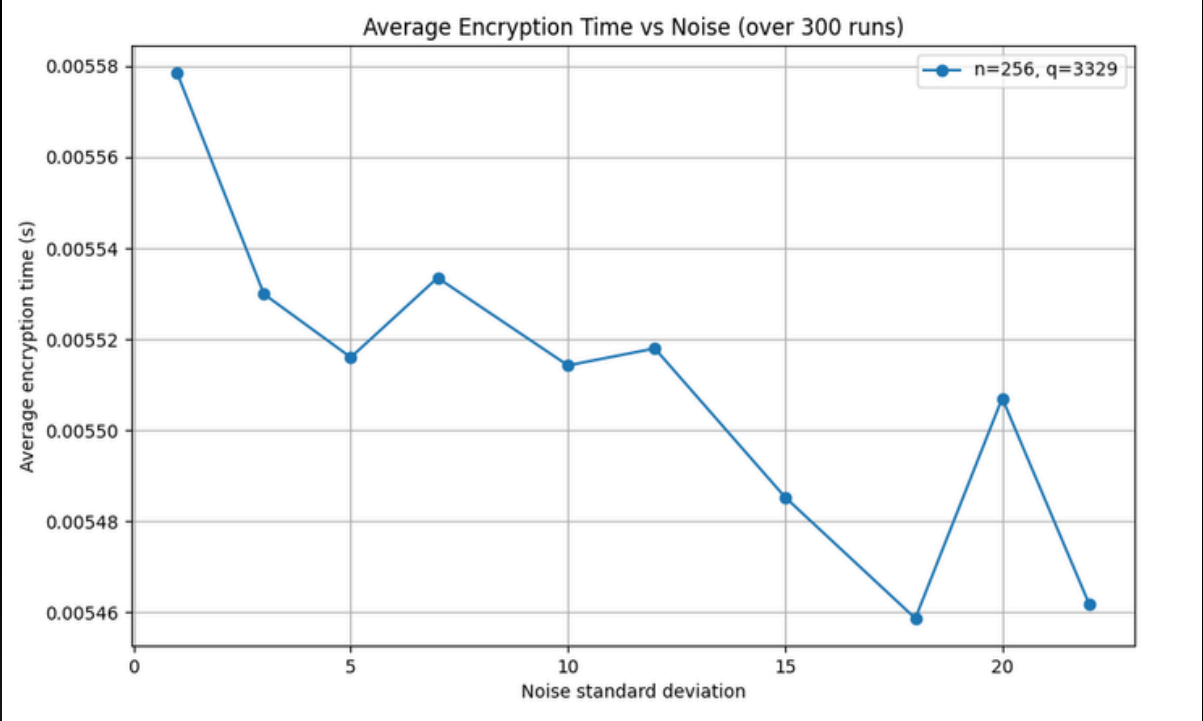
KYBER-512

CRYSTALS – Kyber: a CCA-secure module-lattice-based KEM

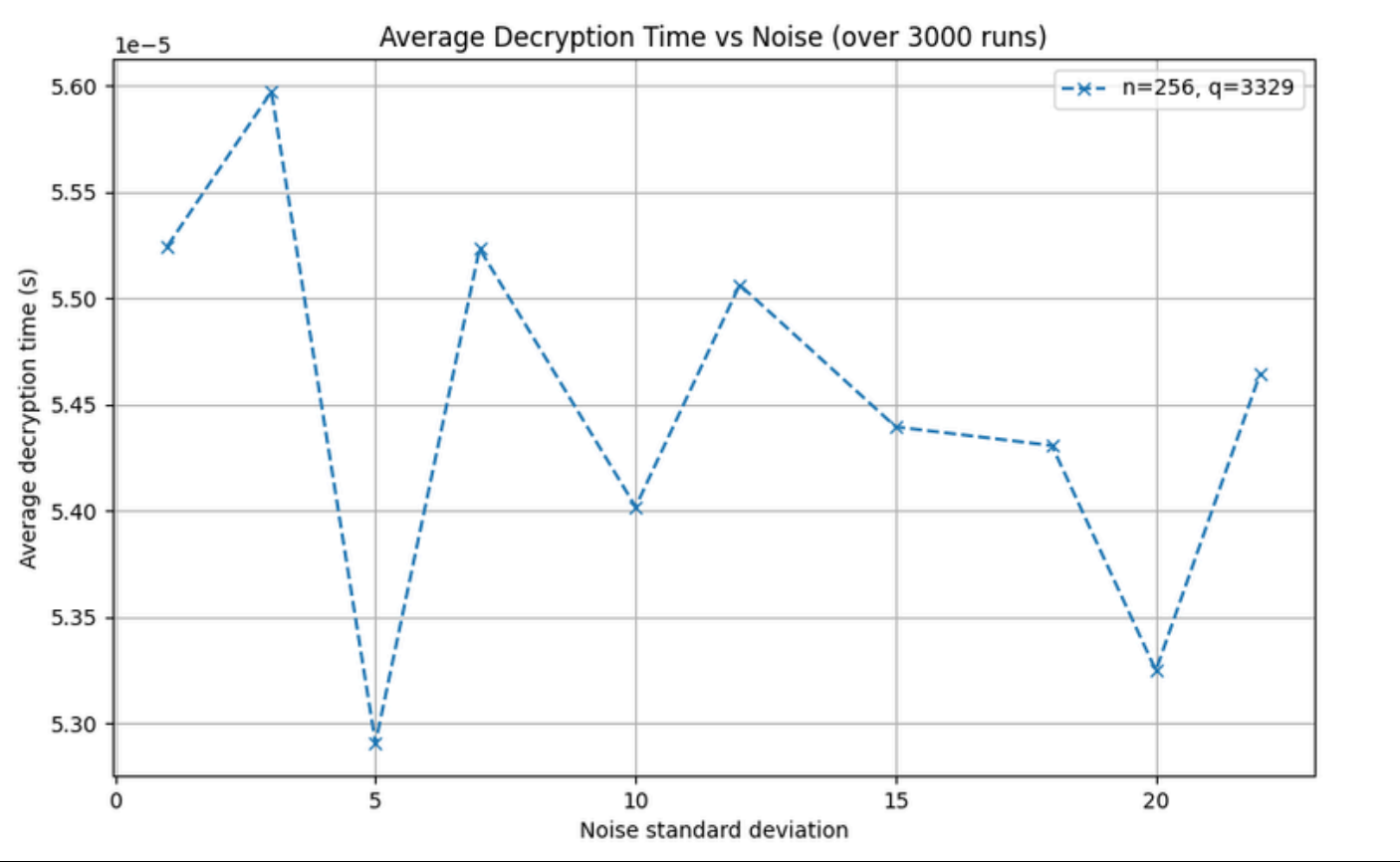
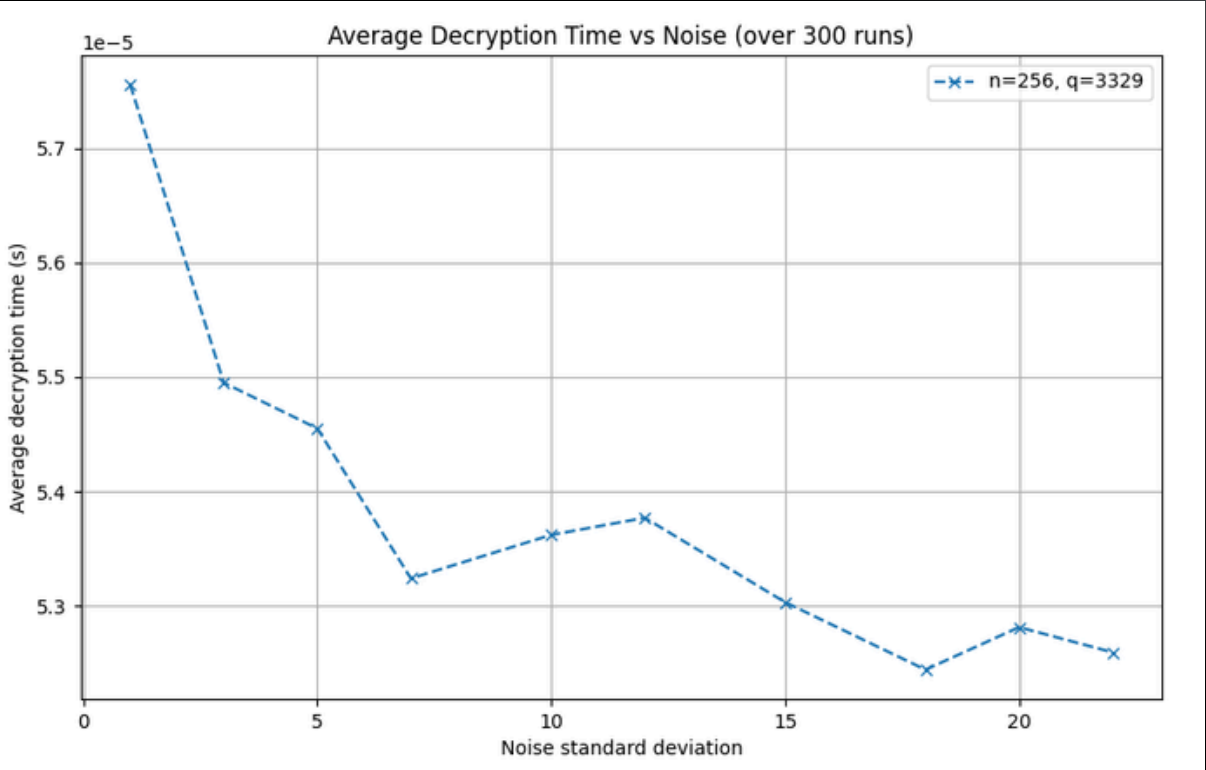
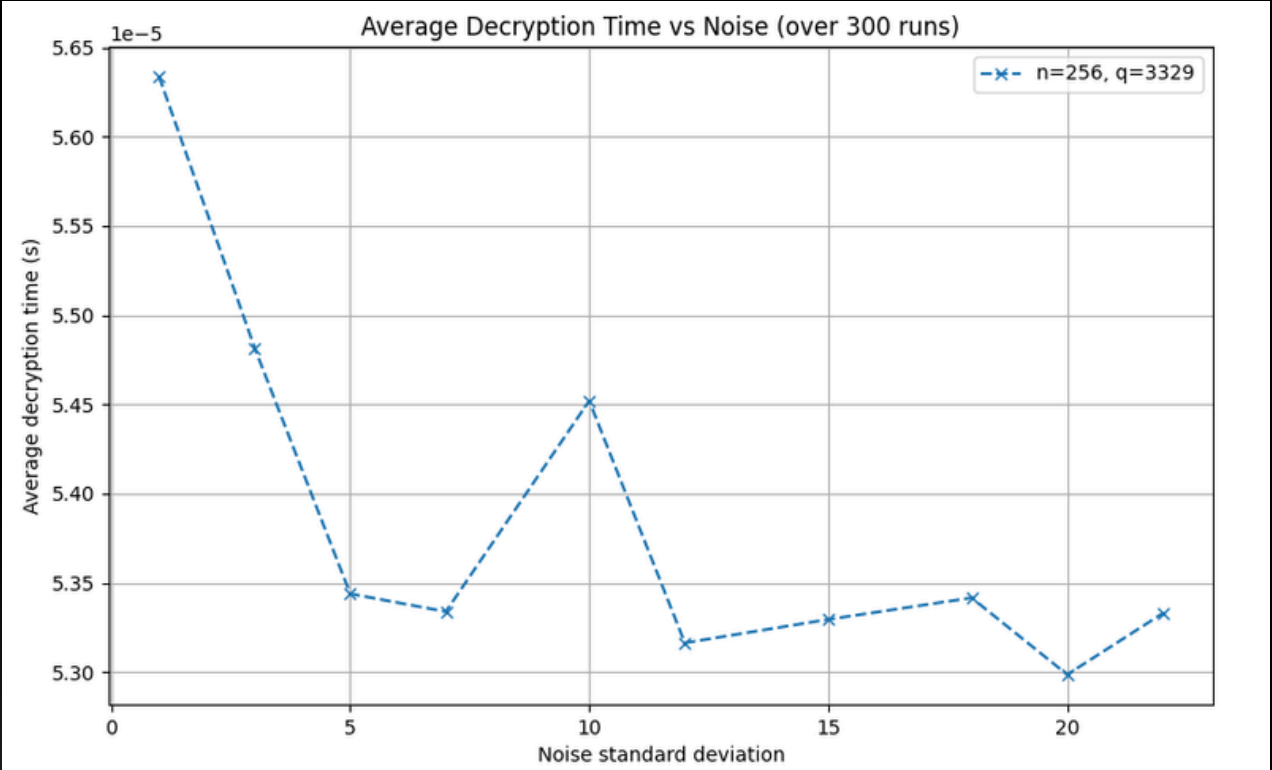
Joppe Bos^{*}, Léo Ducas[†], Eike Kiltz[‡], Tancrede Lepoint[§], Vadim Lyubashevsky[¶],
John M. Schanck^{||}, Peter Schwabe^{**}, Gregor Seiler^{††}, Damien Stehlé^{‡‡},

	Public Key Size (bytes)	Private Key Size (bytes)	Cipher Text Size (bytes)
Kyber512	800	1632	768

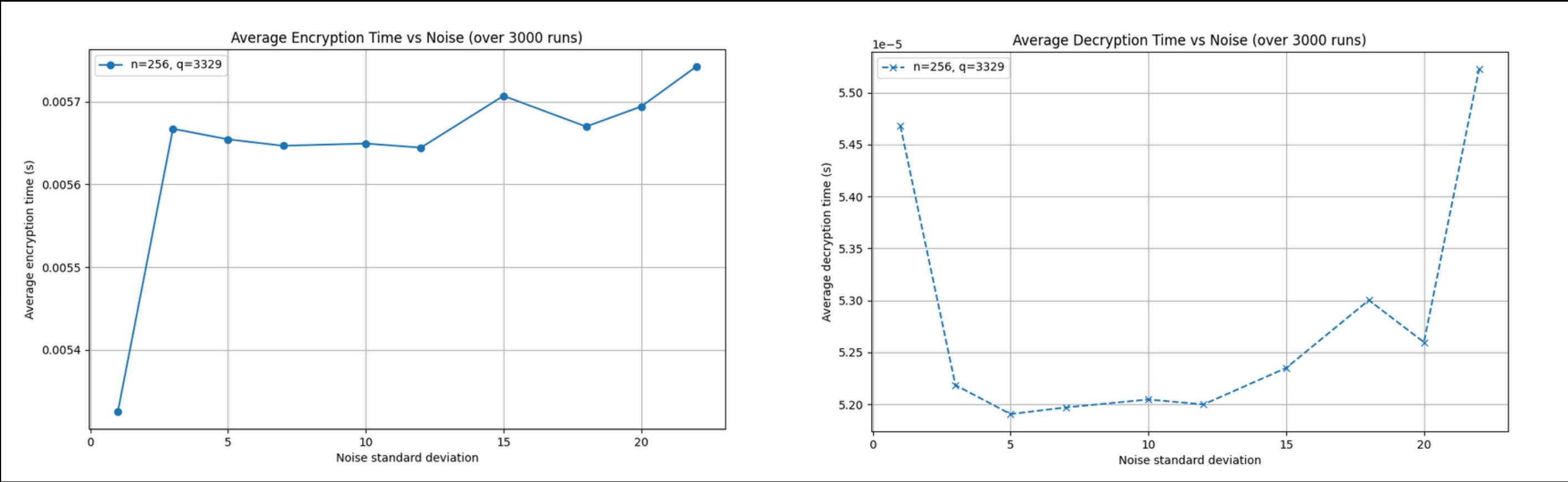
TESTING 2
(ENCODING)



TESTING 2
(DECODING)



TESTING (LIMITATIONS)



CHALLENGES AND LESSONS

CPU Cache bloating

Showed how lattic
encryption worked

RNG normalization

Showed future steps of
cryptography

Memory limitations

Showed limitations of basic
LWE lattice cryptography

Lack of python
knowledge

REFERENCES

Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., & Stehle, D. (2018). Crystals - Kyber: A CCA-secure module-lattice-based KEM. 2018 IEEE European Symposium on Security and Privacy (EuroS&P), 353–367. <https://doi.org/10.1109/eurosp.2018.00032>

Hwang, V. (2024). A survey of polynomial multiplications for lattice-based cryptosystems. IACR Communications in Cryptology. <https://doi.org/10.62056/a0ivr-10k>

Matplotlib.pyplot#. matplotlib.pyplot - Matplotlib 3.5.3 documentation. (n.d.). https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html

Nguyen, D. T., & Gaj, K. (2021). Fast neon-based multiplication for lattice-based NIST Post-quantum cryptography finalists. Lecture Notes in Computer Science, 234–254. https://doi.org/10.1007/978-3-030-81293-5_13

Relyea, R. (2023, October 30). Post-quantum cryptography: Lattice-based cryptography. Red Hat - We make open source technologies for the enterprise. <https://www.redhat.com/en/blog/post-quantum-cryptography-lattice-based-cryptography>



THANK YOU

QUESTIONS?